Active Digital Twin Verification for Robust Federated Learning in IoT Intrusion Detection

Hoang-Hao Pham*†, Phan The Duy *†, Van-Hau Pham*†

*Information Security Laboratory, University of Information Technology, Ho Chi Minh City, Vietnam
†Vietnam National University, Ho Chi Minh City, Vietnam
haoph.16@grad.uit.edu.vn, {duypt, haupv}@uit.edu.vn

***Abstract*—Federated Learning has become a practical approach for training intrusion detection models across distributed Internet of Things devices, but it remains exposed to poisoning attacks, non-IID data heterogeneity, and free-rider exploitation. This paper presents DT-Guard, a defense framework that leverages a server-side Digital Twin as a controlled testing environment for actively verifying client model behavior. Each submitted update is deployed in the Digital Twin and evaluated on synthetic challenge data through a four-layer pipeline that examines detection capability, backdoor resistance, parameter deviation, and cross-round stability. A complementary aggregation scheme called DT-Driven Performance Weighting compares client predictions against the current global model, exposing free-riders whose outputs are nearly indistinguishable from the global baseline. We validate DT-Guard on CIC-IoT-2023 under five poisoning strategies. DT-Guard generally outperforms nine existing defenses in accuracy, false positive rate, and contribution fairness.**

***Index Terms*—Federated learning, Digital twin, Intrusion detection, IoT security, Poisoning attack, Contribution evaluation**

I. Introduction

Federated Learning (FL) allows Internet of Things (IoT) devices to collaboratively train shared models, including Intrusion Detection Systems (IDS), while keeping raw data on-device [1]. This privacy-preserving property is attractive for IoT deployments where transmitting full traffic captures to a central server is neither feasible nor desirable. In practice, however, the data residing on individual IoT nodes tends to be non-independent and non-identically distributed (non-IID) with heavy class imbalance, which already complicates the learning process. This heterogeneity also enables sophisticated poisoning and backdoor attacks [2], [3] that exploit the statistical noise inherent in non-IID settings.

Recent work has attempted to harden FL aggregation against such threats. A comprehensive evaluation in DT-BFL [4] and related studies [5], [6] found that most existing defenses fail against at least one attack type when tested on realistic datasets.

1) **Circumventable defenses.** Modern poisoning attacks, including LIE [2], Min-Max/Min-Sum [3], MPAF [7], and Backdoor, have been specifically crafted to evade existing filters. A cross-method evaluation in DT-BFL [4] shows that each defense, whether Krum [8], Median/Trimmed Mean [9], GeoMed [10], SignGuard [6], ClipCluster [5], or LUP, fails against at least one attack type when tested on the same realistic dataset.
2) **Non-IID confusion.** Passive parameter analysis confuses natural Non-IID deviations with intentional manipulation. SignGuard [6] relies on gradient-sign direction, ClipCluster [5] on norm clipping and clustering, and LUP [4] on MAD-based outlier removal and parameter distance. These criteria make the systems prone to high false positive rates or to missing attacks that have been reverse-optimized, such as LIE [2] and Min-Max/Min-Sum [3].
3) **Missing contribution assessment.** No existing mechanism quantifies the actual knowledge contribution of each client. FedAvg [1] assigns weights by sample count without reflecting update quality. Trust Score in LUP [4] and PoC in PPSG [11] both measure parameter deviation from the global model, which inadvertently rewards free-riders whose updates are nearly identical to the global parameters.

We introduce DT-Guard to address these gaps. The server maintains a Digital Twin (DT) where each client update is deployed and evaluated on synthetic challenge data. A four-layer verification pipeline produces a Trust-Score that determines acceptance. A separate **DT-Driven Performance Weighting (DT-PW)** mechanism compares each client's predictions against the global model's predictions on the same data. Clients that have genuinely learned from local data will disagree with the global model in informative ways; free-riders will produce near-identical outputs and are caught by an automatic *Effort Gate*.

Our contributions are as follows. We characterize how adaptive attacks, non-IID confusion, and the free-rider problem jointly undermine existing FL defenses. We then propose DT-Guard, which uses a server-side Digital Twin for behavioral testing instead of parameter inspection. Within this framework, we design DT-PW, an aggregation mechanism that scores contributions by inference divergence and filters free-riders through an automatic Effort Gate. Finally, we evaluate DT-Guard on CIC-IoT-2023 under five attack strategies against nine baselines.

II. Related Works

A. Background

A *Digital Twin* (DT) is a live software replica of a physical system that stays in sync with the real counterpart [4]. In our setting, the DT runs on the FL server and acts as a sandbox: each client update is loaded into it, tested on challenge data, and scored before the server decides whether to include it in the next global model.

Modern *poisoning attacks* work at either the data or the model level, but the most dangerous variants blend their malicious changes into the normal update distribution so that parameter-level filters miss them. A separate threat is the *free-rider*: a client that simply copies the global model back, contributing nothing new while still receiving the aggregated result. Because its update sits very close to the global parameters, distance-based defenses have no reliable way to flag it.

B. Robust Aggregation Against Poisoning

Classical robust aggregation rules such as Krum [8], coordinate-wise Median, Trimmed Mean [9], and GeoMed [10] filter outliers in parameter space using geometric or order-statistic criteria. While they were effective against first-generation attacks, modern adversaries have learned to stay within the "safe" region. SignGuard [6] introduced gradient-sign analysis to catch directional anomalies, but LIE [2] and Min-Max/Min-Sum [3] explicitly optimize their perturbations to preserve sign consistency. ClipCluster [5] clips norms and then clusters updates, yet it struggles when malicious updates are placed near the centroid of benign clusters. The most sophisticated approach to date, LUP [4], combines MAD-based outlier removal with agglomerative hierarchical clustering and a Trust Score derived from parameter deviation. Even so, its Trust Score mechanism inadvertently favors free-riders, whose updates sit closest to the global model.

All these methods rely on parameter-space analysis. Under strong non-IID conditions, benign updates can naturally vary as much as attack-induced perturbations. This leads to false positives or missed detections. DT-Guard uses behavioral testing: each update is exercised on challenge data in the Digital Twin.

C. Digital Twin in FL and IoT

Several systems have incorporated Digital Twins into FL workflows. DT-BFL [4] builds edge-level DTs that mirror device states and facilitate blockchain interactions. BAFL-DT [12] uses a cloud-based DT for model replication and storage. Zhang et al. [11] integrate DT with asynchronous FL in the PPSG framework, where contribution assessment uses Maximum Mean Discrepancy (MMD) between local and global models. Lu et al. [13] deploy DTs at the edge for model training and aggregation (DITEN).

These systems use the DT for infrastructure or resource management. None of them use the DT as a behavioral testing environment where each client model runs on controlled challenge data before acceptance.

TABLE I: Positioning of DT-Guard among Related Works

Framework	Year	Verif.	DT Role	Contrib.
FedAvg [1]	2017	None	None	Sample
Krum [8]	2017	Passive	None	Select 1
GeoMed [10]	2022	Passive	None	None
SignGuard [6]	2022	Passive	None	None
ClipCluster [5]	2024	Passive	None	Cluster
LUP [4]	2025	Passive	Infra.	Trust
PPSG [11]	2025	Passive	Comp.	PoC
HSDPS [15]	2025	Passive	None	Shapley
DT-Guard	**2026**	**Active**	**Test**	**DT-PW**

D. Contribution Evaluation

FedAvg [1] weights clients by sample count. This approach ignores update quality. Trust Score in LUP and PoC in PPSG both use parameter distance, which rewards free-riders. Shapley value [14] has been applied to blockchain-FL [15] (HSDPS). Computing exact Shapley values has exponential complexity. Approximations introduce noise. The scores influence consensus selection rather than aggregation weights. None of these methods measure actual inference performance on a representative test set to our knowledge.

In summary, Table I positions DT-Guard relative to the works discussed above. Its distinguishing features are active behavioral verification, challenge data generated by deep generative models, and inference-based contribution scoring.

III. Proposed Method

A. System Architecture

DT-Guard aims to produce a robust global intrusion detection model that accurately classifies IoT network traffic, covering both benign flows and multiple attack categories, even when a significant fraction of participating clients submit poisoned or free-rider updates. As shown in Fig. 1, the system involves three entities (IoT Clients, an FL Server, and a server-side Digital Twin Verification Environment) connected by five steps per round:

① **Upload.** Each client trains its local IDS model on the network traffic it observes and uploads the updated parameters $\mathbf{w}_i^{(t)}$ to the FL Server.

② **Deploy.** Rather than aggregating immediately, the server deploys every received update into the DT Sandbox. A Challenge Generator (TabDDPM) supplies synthetic traffic samples $\mathcal{D}_{\text{ch}}$ covering both benign and attack classes (e.g., DDoS, brute force, spoofing). Inside the sandbox, two modules run in parallel: a *Four-Layer Verification Pipeline* that examines detection quality, backdoor resistance, parameter integrity, and cross-round stability to produce a Trust-Score T_i; and a *DT-PW Scoring* module that computes prediction divergence d_i between the client model and the current global model $\mathbf{w}^{(t-1)}$.

③ **Gate & Aggregate.** Both scores feed into a decision gate. Updates that fail the trust or effort threshold are rejected; those that pass proceed to DT-PW Aggregation, which merges accepted updates using contribution-based weights.

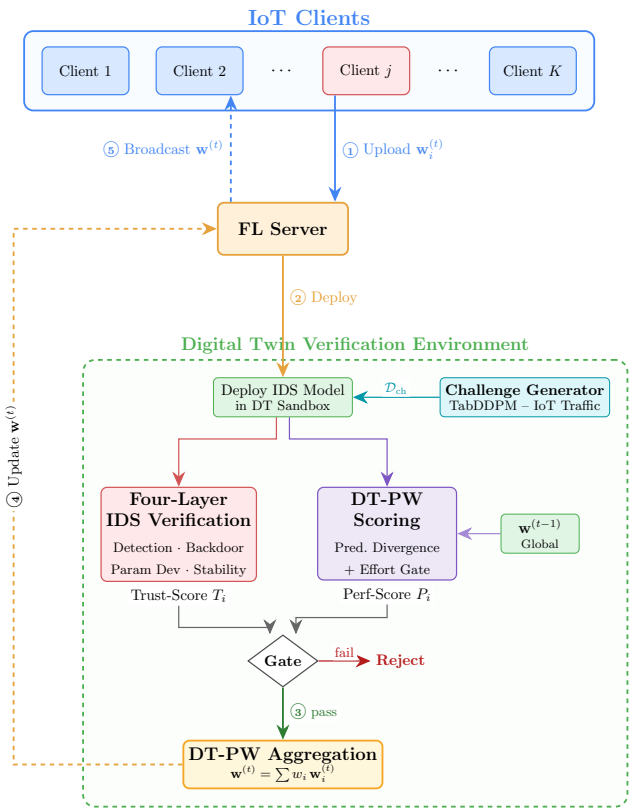

Fig. 1: Overview of the DT-Guard architecture.

④ **Update.** The newly aggregated global model $\mathbf{w}^{(t)}$ is returned to the FL Server.

⑤ **Broadcast.** The server distributes $\mathbf{w}^{(t)}$ back to all clients for local intrusion detection and further training.

Deploying client models into a separate Digital Twin, rather than running them directly on the FL server, provides two critical benefits. First, it isolates potential malicious behavior: a client model with harmful code cannot compromise the server's coordination logic when execution is confined to a sandboxed environment. Second, it allows verification to scale independently: the server can spawn multiple DT instances for parallel testing without affecting aggregation or broadcasting, which keeps the main thread responsive even under heavy load.

B. Challenge Data Generation

Since the FL server has no access to raw client data, it relies on a synthetic generator to produce realistic IoT traffic for the DT. We compare five candidates from two families: *diffusion-based*, including TabDDPM [16] (MLP denoising), TabSyn [17] (VAE + latent diffusion), and ForestDiffusion [18] (XGBoost denoising, CPU-only); and *GAN-based*, including CTGAN [19] (conditional tabular GAN) and WGAN-GP [20] (gradient-penalty GAN). All five are benchmarked on CIC-IoT-2023 across four criterion groups: fidelity, coverage & diversity, conditional control, and efficiency (Fig. 2).

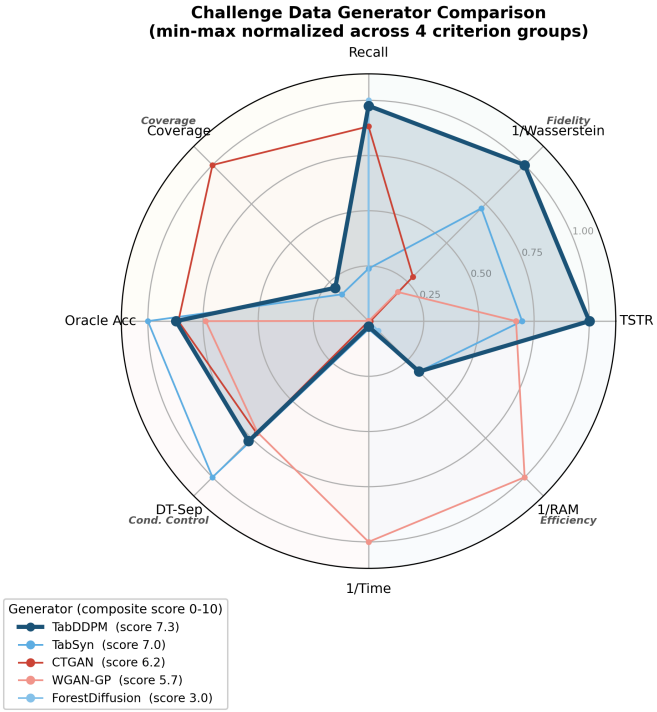

Fig. 2: A/B testing of five challenge data generators across four criterion groups.

TabDDPM achieves the highest composite score (7.3/10, Fig. 2): it leads in fidelity (TSTR 71.4%) and diversity (Recall 0.98) while consuming only 200 s and 25 MB, over 8× faster than CTGAN. TabSyn offers the strongest conditional control but lower recall. CTGAN scores well on coverage yet is too slow for per-round use. ForestDiffusion cannot produce class-conditional samples, and WGAN-GP generates the least diverse data. We therefore adopt TabDDPM as the default challenge generator.

C. Four-Layer Behavioral Verification

Every candidate update $\mathbf{w}_i^{(t)}$ is loaded into the DT and evaluated on $\mathcal{D}_{\text{ch}}$ through four complementary checks. The mechanism is controlled by hyperparameters: $\alpha \in [0,1]$ balances performance vs. parameter similarity; $\lambda_{\text{bd}}, \lambda_{\text{stab}} \in \mathbb{R}^+$ are penalty coefficients; $D_{\max} > 0$ normalizes parameter distance; and $\theta_{\text{div}} > 0$ is the divergence threshold for strict mode.

a) Layer 1: IDS Performance: The model is evaluated on the challenge set using binary classification (benign vs. attack). The F1-score serves as the primary quality signal:

$$s_i^{\text{ids}} = \text{F1}\big(\mathbf{w}_i^{(t)},\ \mathcal{D}_{\text{ch}}\big) \in [0,1]. \tag{1}$$

b) Layer 2: Backdoor Resistance: Rather than measuring ASR directly (which requires known trigger labels), the DT monitors the variance of prediction confidence across the challenge set. A clean model produces stable confidence;

a backdoor-infected model shows abnormally high variance when triggered samples are present. The penalty is:

$$\delta_i^{\text{bd}} = \begin{cases} \lambda_{\text{bd}}, & \text{if } \sigma\big(\text{conf}(\mathbf{w}_i^{(t)}, \mathcal{D}_{\text{ch}})\big) > \tau_{\text{bd}}, \\ 0, & \text{otherwise}, \end{cases} \tag{2}$$

where $\sigma(\cdot)$ is the standard deviation of the model's softmax confidence on $\mathcal{D}_{\text{ch}}$ and τ_{bd} is the confidence variance threshold.

c) Layer 3: Parameter Deviation: We compute the mean per-layer ℓ_2 distance between the client update and the global model, normalized by $D_{\max}$:

$$\Delta_i^{(t)} = \frac{1}{L} \sum_{\ell=1}^{L} \frac{\|\mathbf{w}_{i,\ell}^{(t)} - \mathbf{w}_{\ell}^{(t-1)}\|_2}{D_{\max}} \geq 0, \tag{3}$$

where L is the number of trainable layers. This is mapped to a similarity score:

$$s_i^{\text{param}} = 1 - \min\big(1,\ \Delta_i^{(t)}\big) \in [0, 1]. \tag{4}$$

When $\Delta_i^{(t)} > 1$, the parameters diverge beyond the normalization limit, forcing $s_i^{\text{param}} = 0$.

d) Layer 4: Cross-Round Stability: Erratic behavior over time is a warning sign. The DT tracks each client's false positive rate over a sliding window of τ recent rounds. If the variance exceeds a threshold, a penalty is applied:

$$\delta_i^{\text{stab}} = \begin{cases} \lambda_{\text{stab}}, & \text{if } \text{Var}\big(\text{FPR}_i^{(t-\tau)}, \ldots, \text{FPR}_i^{(t)}\big) > \tau_{\text{stab}}, \\ 0, & \text{otherwise}, \end{cases} \tag{5}$$

where τ_{stab} is the FPR variance threshold.

e) Trust-Score Fusion: The four layers are combined into a single Trust-Score using an adaptive aggregation function that depends on parameter divergence severity:

$$T_i = f(s_i^{\text{ids}}, s_i^{\text{param}}; \Delta_i^{(t)}) - \delta_i^{\text{bd}} - \delta_i^{\text{stab}}, \tag{6}$$

where the base score aggregation function $f(\cdot)$ adapts to divergence level:

$$f(\cdot) = \begin{cases} \alpha s_i^{\text{ids}} + (1-\alpha) s_i^{\text{param}}, & \Delta_i^{(t)} \leq \theta_{\text{div}}, \\ \min(s_i^{\text{ids}}, s_i^{\text{param}}), & \Delta_i^{(t)} > \theta_{\text{div}}. \end{cases} \tag{7}$$

In balanced mode (first case), Layers 1 and 3 contribute proportionally to α, favoring IDS performance. In strict mode (second case), both layers must score high—requiring strong IDS performance *and* low parameter deviation. A client is accepted only when $T_i \geq \theta_T$, where θ_T is the trust threshold.

D. DT-Driven Performance Weighting

Once a client passes the trust gate, DT-PW quantifies its contribution by comparing its predictions to those of the current global model on the same challenge data.

Prediction divergence. Both $\mathbf{w}_i^{(t)}$ and $\mathbf{w}^{(t-1)}$ are loaded into the DT. Their prediction vectors on $\mathcal{D}_{\text{ch}}$ are compared:

$$d_i = \frac{1}{M} \sum_{j=1}^{M} \mathbb{1}\big[\hat{y}_{i,j} \neq \hat{y}_{g,j}\big]. \tag{8}$$

Effort Gate. Free-riders contribute nothing. We set an adaptive threshold to flag such behavior:

$$\theta_E = \mu_d - \beta \cdot \sigma_d, \tag{9}$$

where μ_d and σ_d are the mean and standard deviation of $\{d_i\}$ across accepted clients. The parameter β controls sensitivity. If $d_i < \theta_E$, the client's contribution weight is zeroed out.

Performance-Score and final aggregation. For clients that pass the Effort Gate ($d_i \geq \theta_E$), we combine detection capability with contribution novelty:

$$P_i = s_i^{\text{ids}} \cdot \frac{d_i}{\max_k\ d_k}. \tag{10}$$

The aggregation weight merges Trust-Score and Performance-Score:

$$w_i = \frac{T_i \cdot P_i}{\sum_{k \in \mathcal{A}} T_k \cdot P_k}, \tag{11}$$

where $\mathcal{A}$ is the set of accepted, non-free-rider clients. The new global model is computed as:

$$\mathbf{w}^{(t)} = \sum_{i \in \mathcal{A}} w_i\ \mathbf{w}_i^{(t)}. \tag{12}$$

IV. Experiments and Evaluation

A. Environmental Setup

We use CIC-IoT-2023 [21], a real-world IoT traffic dataset with seven attack categories and benign traffic (34 classes, 39 features). The dataset is partitioned across K=20 clients via Dirichlet-based non-IID sampling (α=0.5).

The local model, IoTAttackNet, is a fully connected network (39→256→128→64→34) with ReLU activation, BatchNorm, and Dropout (p=0.3). Training uses Adam (lr=10^{-3}), weighted Cross-Entropy, batch size 512, E=3 local epochs, and R=20 rounds. For challenge data generation, we compare five candidates: TabDDPM [16] (256 hidden units, SiLU, T=200 steps, 100 epochs), TabSyn [17] (VAE + latent diffusion, 256 hidden units, 100 epochs), ForestDiffusion [18] (XGBoost-based, T=200, CPU-only), CTGAN [19] (256 hidden units, 300 epochs), and WGAN-GP [20] (256 hidden units, gradient penalty λ=10, 300 epochs). The trust threshold is θ_T=0.6.

We evaluate five attacks: Backdoor, LIE [2], Min-Max, Min-Sum [3], and MPAF [7] at 50% malicious ratio, against nine baselines: FedAvg [1], Krum [8], Median, Trimmed Mean [9], GeoMed [10], SignGuard [6], ClipCluster [5], LUP [4], and PoC [11].

B. Defense Robustness

Fig. 3 reports results at 50% malicious clients, the hardest ratio we tested. In panel (a), DT-Guard averages 72.8% accuracy and stays between 72.5% and 73.1% regardless of attack type. LUP is close at 72.2% overall, though it dips to 70.3% under Backdoor. SignGuard averages 68.0% but is less consistent, dropping to 53.7% under MPAF. The other defenses all have at least one attack that breaks them below 2.3%: GeoMed under LIE, PoC under LIE and Min-Sum, ClipCluster under MPAF, FedAvg and Krum under Min-Sum and MPAF.

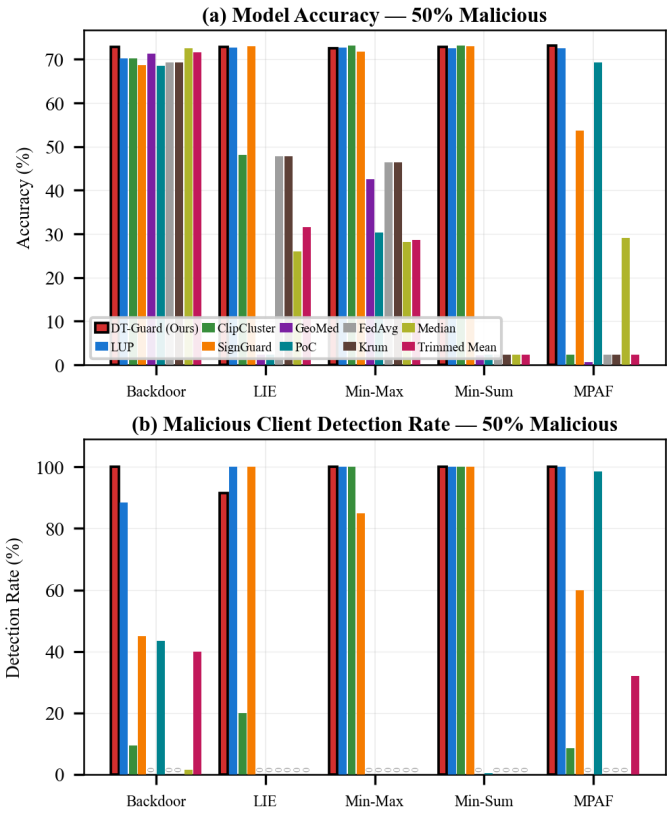

Fig. 3: Defense comparison at 50% malicious clients. (a) Model accuracy. (b) Detection rate.

The detection rates in panel (b) tell a related story. DT-Guard catches 98.3% of attackers on average—100% on four of five attacks, 91.5% on LIE. LUP reaches 97.7% but misses 11.5% of Backdoor attackers. SignGuard averages 78.0%; its weak spot is Backdoor at 45.0%. ClipCluster and PoC both fall below 50% on average. FedAvg, Krum, and GeoMed never reject anyone, so their detection rate is 0%. Overall, DT-Guard is the only defense that simultaneously maintains the highest accuracy and the highest detection rate across all five attack types, confirming that behavioral verification generalizes where parameter-based filters do not.

C. False Positive Analysis

Fig. 4 shows how often each defense wrongly rejects honest clients under the same scenario. DT-Guard maintains 0.0% FPR across all five attack types—the only active defense to do so while still reaching the highest detection rate (98.3%, Fig. 3b). Every other defense that actively rejects clients trades detection for false alarms.

LUP reaches 97.7% detection but produces 26.7% average FPR, peaking at 49.5% under Backdoor: nearly half of honest clients are excluded. PoC is worse, with 74.5% FPR under LIE and 39.5% on average. Trimmed Mean averages 25.6% FPR; GeoMed hits 81.0% under Min-Sum. ClipCluster keeps FPR at 2.3% on average, but at the expense of detecting only 47.6% of attackers.

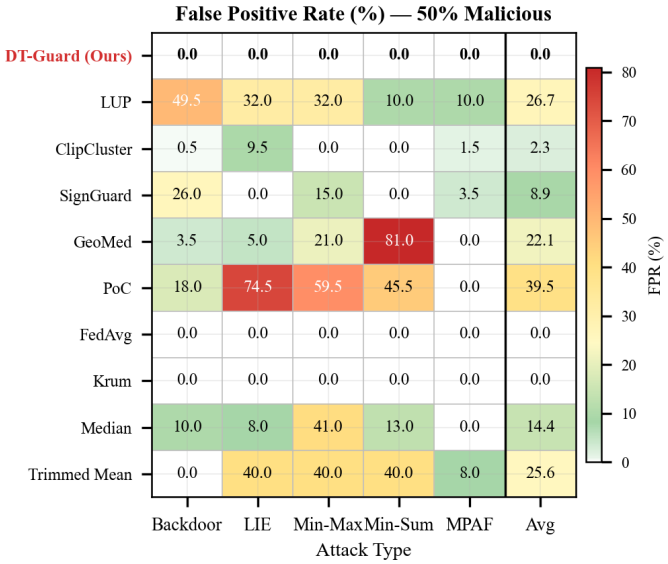

Fig. 4: False positive rate (%) at 50% malicious clients.

FedAvg and Krum report 0% FPR because they never reject any client. Their 0% detection rate (Fig. 3b) confirms they simply accept everything, including malicious updates.

The reason DT-Guard escapes this tradeoff is that it judges models by what they *do* on challenge data, not by how their parameters look. A honest client with non-IID data may have unusual parameters, but it still classifies traffic correctly—so the four-layer pipeline lets it pass. Parameter-based filters cannot make this distinction.

D. Contribution Fairness

We inject 4 free-rider clients (20%) that copy the global model with negligible noise instead of training. Fig. 5 compares four weighting strategies. **DT-PW** correctly assigns weight 0.0 to all four free-riders and redistributes their share equally among the 16 genuine clients (weight 0.0625 each, i.e., $1/16$), achieving the highest accuracy of 72.97%.

Trust-Score [4] produces the opposite effect: because it scores clients by parameter proximity to the global model, free-riders (whose updates are nearly identical to the global model) receive a disproportionately high weight of 0.2448, while genuine clients are suppressed to just 0.0013. This weight inversion degrades accuracy to 46.49%. **Uniform** and **FedAvg** assign equal weights (0.05) to all clients regardless of contribution, reaching 72.51%—comparable to DT-PW only because free-riders echo the global model in this scenario. The Effort Gate (Eq. 9) drives DT-PW's advantage by zeroing clients whose predictions nearly match the global model.

E. Computational Overhead

DT-Guard adds 80.8 ms per round (less than 1% of total round time), decomposed into DT verification and DT-PW scoring. This overhead is negligible compared to the multi-minute training rounds typical in IoT FL deployments. Peak memory is 485 MB, the lowest among all ten strategies,

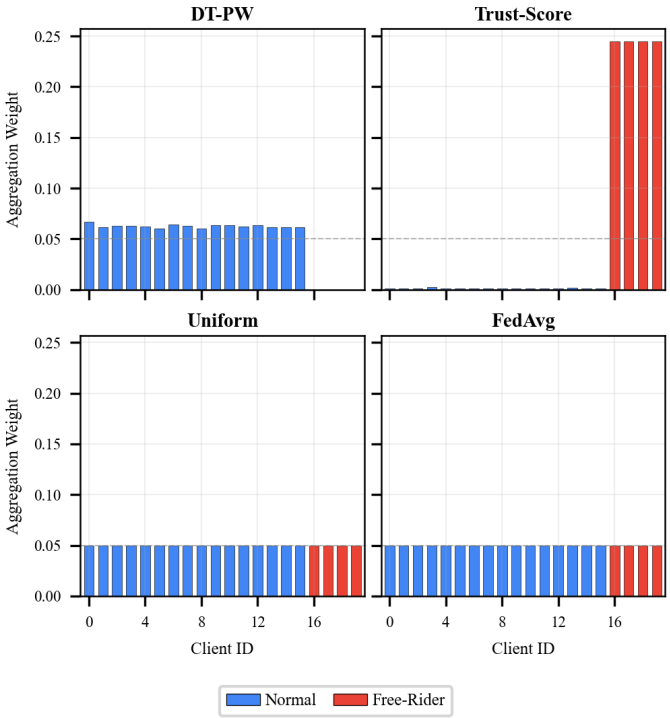

Fig. 5: Per-client aggregation weights averaged over the last 5 of 20 rounds.

because the sandbox loads one model at a time and frees resources before the next. Source code and reproduction scripts are available at https://github.com/HaoPhamUIT/DT-Guard-FL.

V. Conclusion and Future Work

DT-Guard replaces passive parameter inspection with active behavioral testing in federated learning. By loading client updates into a Digital Twin and evaluating them on synthetic challenge data, the four-layer verification pipeline distinguishes genuine non-IID variation from poisoning based on what models *do* rather than parameter appearance. The DT-PW aggregation mechanism further addresses free-riding by comparing client predictions with the global model and filtering out contributions that add no new knowledge.

Experimental evaluation on CIC-IoT-2023 demonstrates that DT-Guard achieves superior performance compared to nine baseline defenses. The framework maintains high accuracy and detection rates while keeping false positives near zero. Furthermore, DT-PW successfully identifies and suppresses all free-riders, whereas proximity-based aggregation methods inadvertently assign them high weights. The computational overhead remains acceptable for practical deployment.

Future research directions include validating DT-Guard on additional IoT security datasets, developing adaptive threshold mechanisms for dynamic attack scenarios.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR, 2017, pp. 1273–1282.

[2] G. Baruch, M. Baruch, and Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[3] V. Shejwalkar and A. Houmansadr, "Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proceedings of the 28th Network and Distributed System Security Symposium (NDSS)*, 2021.

[4] W. Issa, N. Moustafa, B. Turnbull, and K.-K. R. Choo, "DT-BFL: Digital twins for blockchain-enabled federated learning in Internet of Things networks," *Ad Hoc Networks*, vol. 178, p. 103934, 2025.

[5] S. Li, E. C.-H. Ngai, and T. Voigt, "An experimental study of Byzantine-robust aggregation schemes in federated learning," *IEEE Transactions on Big Data*, vol. 10, no. 6, pp. 975–988, 2024.

[6] J. Xu, S.-L. Huang, L. Song, and T. Lan, "SignGuard: Byzantine-tolerant federated learning through collaborative malicious gradient filtering," in *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2022, pp. 530–540.

[7] X. Cao and N. Z. Gong, "MPAF: Model poisoning attacks to federated learning based on fake clients," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2022, pp. 3396–3404.

[8] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[9] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 5650–5659.

[10] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Transactions on Signal Processing*, vol. 70, pp. 1142–1154, 2022.

[11] Z. Zhang, H. Peng, L. Li, and S. Bao, "Adaptive asynchronous federated learning for digital twin driven smart grid," *IEEE Transactions on Smart Grid*, vol. 16, no. 5, pp. 4167–4181, 2025.

[12] Y. Qu, S. R. Pokhrel, S. Nepal, L. Gao, and Y. Xiang, "Decentralized privacy using blockchain-enabled federated learning in fog computing," *IEEE Internet of Things Journal*, vol. 8, no. 8, pp. 6521–6533, 2021.

[13] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Communication-efficient federated learning and permissioned blockchain for digital twin edge networks," *IEEE Internet of Things Journal*, vol. 8, no. 4, pp. 2276–2288, 2020.

[14] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games II*, H. W. Kuhn and A. W. Tucker, Eds. Princeton University Press, 1953, pp. 307–317.

[15] X. Zhang, H. Liu, and F. Yang, "Enhanced traffic carbon emissions data sharing and modeling via blockchain and personalized federated learning in IIoT ecosystem," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 31 064–31 078, 2025.

[16] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 17 564–17 579.

[17] H. Zhang, J. Zhang, B. Srinivasan, Z. Shen, X. Qin, and C. Faloutsos, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[18] A. Jolicoeur-Martineau, K. Fatras, and T. Kachman, "Generating and imputing tabular data via diffusion and flow-based gradient-boosted trees," in *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR, 2024, pp. 1288–1296.

[19] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[20] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of Wasserstein GANs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[21] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.